

AirTwinNet – Required Tools & Technology Stack

A Digital Twin Enabled Multi-Hybrid Framework Combining Satellite Imagery, IoT Sensors, Graph Neural Networks, and Explainable AI for Urban Air Quality Prediction

1. Development Environment

Category	Tool / Technology	Purpose
Programming	Python 3.11	Main programming language
Environment	Conda	Environment and dependency management
IDE	VS Code	Main development environment
Experimentation	Jupyter Notebook	EDA, experiments and model development
Version Control	Git + GitHub	Version control and backup

2. Data Science & Processing

NumPy, Pandas, Matplotlib, Seaborn and SciPy for numerical computation, data manipulation, analysis and visualization. Parquet for efficient data storage.

3. Satellite & Geospatial

GeoPandas, Rasterio, Shapely, PyProj, Xarray and rioxarray for geospatial and satellite-data processing. Folium/Plotly for spatial visualization.

4. Machine Learning

Scikit-learn for preprocessing, baseline models and evaluation; XGBoost for strong tabular baseline modeling.

5. Deep Learning & Graph ML

PyTorch for deep learning and PyTorch Geometric (PyG) for graph construction and GCN/GAT/GraphSAGE-style learning. ST-GNN will be developed after the data pipeline is stable.

6. Explainable AI

SHAP for feature/prediction explanations; Captum can be used when PyTorch-specific interpretability is required.

7. Digital Twin

A Python-based state and synchronization layer representing urban conditions, sensor observations, pollution state and environmental conditions. No heavy 3D engine is required initially.

8. Database & Storage

CSV/Parquet initially; PostgreSQL + PostGIS later for structured spatial storage and queries.

9. Backend & Dashboard

FastAPI for APIs and Streamlit for the initial research dashboard containing AQI, forecasts, maps, hotspots, Digital Twin state and XAI.

10. Testing & Code Quality

pytest for testing; logging for application logs; Ruff for linting; Black for formatting; python-dotenv for environment configuration.

11. Deployment

Docker for containerization and reproducible deployment. Cloud deployment will be handled in the final phase.

12. Recommended Final Stack

Layer	Technology
Language & Environment	Python 3.11 + Conda
Development	VS Code + Jupyter + Git/GitHub
Data	NumPy + Pandas + Parquet
Visualization	Matplotlib + Seaborn + Plotly
Geospatial	GeoPandas + Rasterio + Shapely + PyProj + Xarray
ML	Scikit-learn + XGBoost
Deep Learning	PyTorch
Graph ML	PyTorch Geometric
XAI	SHAP + Captum (if required)
Digital Twin	Python-based state/synchronization layer
Database	PostgreSQL + PostGIS (later)
Backend	FastAPI
Dashboard	Streamlit
Testing & Quality	pytest + Ruff + Black + logging
Deployment	Docker

Installation Strategy: Install dependencies phase-by-phase rather than all at once to avoid conflicts, especially around PyTorch, PyTorch Geometric and geospatial libraries.